

National University of Computer and Emerging Sciences



Laboratory Manuals

for

Data Structures Lab

(CL -2001)

Course Instructor	Mr. Waqas Rana
Lab Instructor	Ms. Fatima Ali
Lab TA	Mr. Abdur Rehman
Section	BSE-3B
Semester	Fall 2026

*Department of Computer Science
FAST-NU, Lahore, Pakistan*

Lab Manual 03

Objectives:

- Understand the concept of a Doubly Linked List (DLL) and a Circular Linked List (CLL), including the Circular Doubly Linked List variant.
- Learn how to convert a plain Singly Linked List into a Circular Linked List.
- Learn how to solve difficult level problems on doubly and circular linked lists using plain pointers.
- Implement a custom Iterator class enabling STL-style, range-based traversal of a circular linked list.

Doubly Linked List Implementation in C++

1. Background

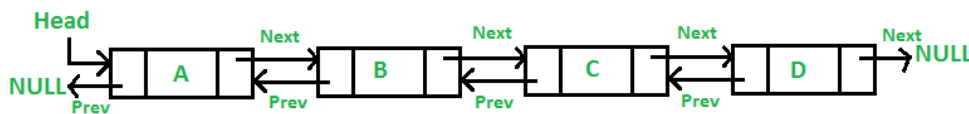
A Singly Linked List allows traversal in only one direction, since each node stores a link only to the next node. This makes some operations, such as moving backward or deleting the last node, less efficient.

A **Doubly Linked List (DLL)** extends this idea by giving each node two links:

- next → points to the next node
- prev → points to the previous node

This bidirectional linkage makes traversal and updates more flexible. The structure is usually managed with two special pointers:

- head → points to the first node
- tail → points to the last node



Advantages of Doubly Linked List

- **Two-way Traversal:** Can move both forward and backward.
- **Efficient Deletion:** Direct access to the predecessor allows easier removal of nodes.
- **Fast Insertions at Both Ends:** With head and tail pointers, insertions at front and back are $O(1)$.
- **Versatility:** Useful for implementing Deques, navigation systems, and other complex data structures.

Disadvantages of Doubly Linked List

- **More Memory:** Each node stores an extra pointer (prev).
- **Extra Complexity:** Both pointers (next and prev) must be updated correctly during insertions/deletions.
- **Higher Overhead:** Slight increase in memory and runtime due to managing two links.
- **Error-Prone:** Forgetting to update either pointer can easily break the list.

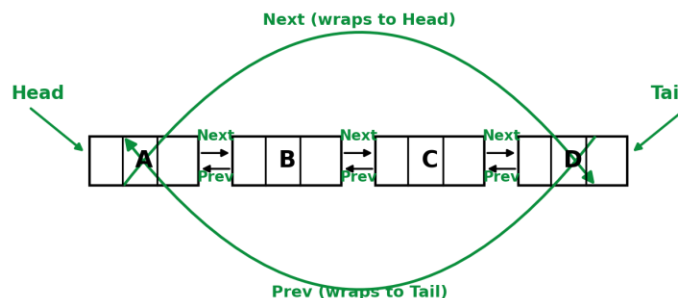
Circular Linked List Implementation in C++

1. Background

A **Circular Linked List (CLL)** is a variation of a linked list where the last node points back to the first node instead of storing NULL. This circular connection allows traversal to wrap around the list continuously.

There are two main types:

- **Singly Circular Linked List** → Each node has only a next pointer, and the last node's next points to head.
- **Doubly Circular Linked List** → Each node has both next and prev pointers, forming a circular loop in both directions.



Advantages of Circular Linked List

- Traversal can start from any node and continue indefinitely.
- Efficient for applications that require repeated cycling (e.g., round-robin scheduling).
- Saves a NULL reference compared to linear lists.
- Insertion at the end or beginning can be done quickly if tail is maintained.

Disadvantages of Circular Linked List

- Traversal can fall into infinite loops if not handled carefully.
- Slightly more complex insertion and deletion logic compared to singly lists.

- Searching is still $O(n)$.

Best Practices

- Handle empty list cases before any operation.
- Maintain a tail pointer for efficient end insertions/deletions ($O(1)$ instead of $O(n)$).
- Use do-while loops for traversal to ensure the head node is not skipped.
- Stop traversal when back at head to avoid infinite loops.
- Handle single-node lists separately (deletion must reset head and tail to NULL).
- Update head and tail consistently after insertions/deletions.
- Encapsulate repeated logic (e.g., linking head $\leftrightarrow$ tail) into helper functions.
- Minimize unnecessary traversals (use tail for direct access).
- Use clear stopping conditions (while (current != head)).
- Choose between Singly and Doubly CLL wisely (memory vs. flexibility).
- Free memory properly when deleting nodes or destroying the list.

What is an Iterator?

An **iterator** is an object that behaves like a **pointer** and allows you to **traverse** elements of a data structure (list, array, tree, etc.) one by one. Think of it as a **cursor** that moves forward/backward across elements. Iterators abstract away the underlying structure (array indexing, pointer navigation).

Without iterators, traversing a linked list would require manually managing raw pointers like:

```
Node* temp = head;
while (temp) {
    cout << temp->data;
    temp = temp->next;
}
```

With iterators, this becomes simpler, safer, and uniform:

```
for (Iterator it = playlist.begin(); it != playlist.end(); ++it) {
    cout << *it;
}
```

Iterators **improve abstraction**, the user doesn't care how data is stored; they just traverse.

Iterator Best Practices

- Always compare iterators to end(), not nullptr.
- Use ++it instead of it++ (slightly faster).
- Be careful: after deleting an element, its iterator becomes invalid.
- Iterators make code generic → same loop works for any data type.

Advantages of Iterators

1. Cleaner traversal syntax : Instead of writing node = node->next, students just use ++it or --it.
2. Encapsulation: Users don't need to know the details of the Node structure.
3. Bidirectional movement: In a DLL, iterators naturally allow both forward and backward traversal.
4. Consistency with STL: Makes custom containers feel like std::list or std::vector, so students practice in a real-world style.

Exercises (25)

This is your first lab building a Doubly Linked List, so you will construct it from scratch and then extend it into a Circular Doubly Linked List (the tail's next points to the head, and the head's prev points to the tail). Once your Circular Doubly Linked List is working,

1. Write a menu-driven C++ program that simulates a music playlist with the following features: **(1)**

```
==== Circular Playlist Manager ====
1. Add Song at Beginning
2. Add Song at End
3. Add Song at Position
4. Remove Song from Beginning
5. Remove Song from End
6. Remove Song from Position
7. Search Song
8. Traverse Forward
9. Traverse Backward
10. Play N Songs on Loop
11. Remove Duplicate Songs
12. Find Song N from End
13. Check Playlist Palindrome
14. Exit
```

2. Add a Song (3)

Implement insertion into the circular playlist:

- At the beginning
- At the end
- At a specific position

3. Remove a Song (3)

Implement deletion from the circular playlist:

- From the beginning
- From the end
- From a specific position

4. Search for a Song (2)

Check if a given song exists anywhere in the playlist. Because the list is circular, your search must stop after one full revolution instead of running forever.

5. Implement a Custom Iterator (5)

Add a custom Iterator class to your Circular Doubly Linked List so the playlist can be traversed in both directions, the same way STL containers are. Your Iterator must support:

- `operator++` (prefix) → advances the iterator to the next song
- `operator--` (prefix) → moves the iterator to the previous song
- `operator*` → returns a reference to the current song
- `operator!=` → compares two iterators so a loop knows when to stop

Add `begin()` / `end()` member functions to your playlist class for forward traversal, and `rbegin()` / `rend()` for backward traversal, all returning Iterator objects. Since the list is circular, `end()` and `rend()` must mark the point where traversal completes one full revolution, not a null pointer, otherwise a loop will never terminate. Use your iterator to implement `printPlaylistUsingIterator()`, which prints every song using a range-based for loop.

Example:

```
Playlist = [S1, S2, S3, S4]
for (auto& song : playlist) cout << song << " ";
Output = S1 S2 S3 S4
```

6. Traverse Playlist Forward and Backward (2)

Write a function that traverses the playlist forward (from head to tail) and backward (from tail to head) using your custom iterator, not plain pointers. Display each song's title in order.

Example:

Playlist = [S1, S2, S3, S4]
Forward = S1 -> S2 -> S3 -> S4
Hint: use begin()/end() and the ++ operator of your iterator class.

Playlist = [S1, S2, S3, S4]
Backward = S4 -> S3 -> S2 -> S1
Hint: use rbegin()/rend() and the -- operator of your iterator class.

7. Play N Songs on Loop (2)

Because the underlying list is circular, playback should continue indefinitely with no extra "circular mode" flag needed. Implement a Play feature that starts at the current song pointer and plays the next n songs, wrapping around to head automatically whenever it passes tail.

Example:

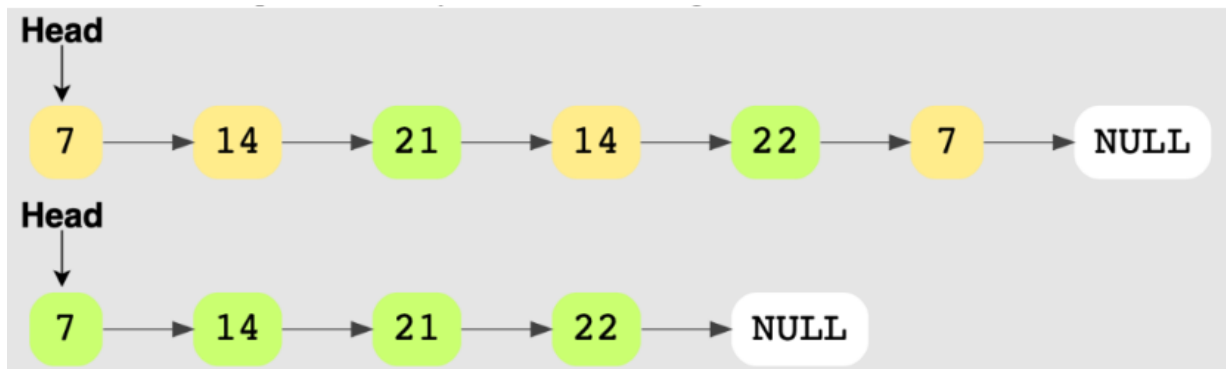
Playlist = [S1, S2, S3], n = 7
Output = S1 -> S2 -> S3 -> S1 -> S2 -> S3 -> S1

8. Remove the Duplicate Songs from the Playlist (3)

Implement a function that scans the circular playlist and removes any duplicate song titles, keeping only the first occurrence. An illustration is also provided for your understanding.

Example:

Playlist = [S1, S2, S1, S3, S2]
After removing duplicates = [S1, S2, S3]



9. Find the Song N Positions from the End (3)

Implement nthFromEnd(int n), which returns the song that is n positions before the last song in the playlist (n = 1 means the last song, n = 2 means the second-last, and so on).

Example:

Playlist = [S1, S2, S3, S4, S5], n = 2
Output = S4

10. Check if the Playlist Reads the Same Forwards and Backwards [Bonus (if time allows)(2)]

A palindrome is any sequence that reads the same from both ends. Implement `isPalindrome()`, which checks whether the sequence of song titles in the playlist is a palindrome for example, a playlist built to loop symmetrically for a themed party.

Example:

```
Playlist = [S1, S2, S3, S2, S1] -> Palindrome  
Playlist = [S1, S2, S3, S4] -> Not a Palindrome
```

11. Exit Program (1)

Submission Guidelines

1. submit following the only file strictly following the naming convention
 - a. l261234.cpp